

第10週: エンジンの換装 (Next.js 移行)

～ Vanilla JS から React/Next.js へ ～

1. なぜ React / Next.js なのか？

- **コンポーネントベース:** UI を部品化して再利用できる。
- **宣言的 UI:** 「どう動かすか」ではなく「どうあるべきか」を書く。
- **エコシステム:** 世界中の便利なライブラリが使える。

これまでの Vanilla JS での苦労（DOMの直接操作）が、いかに自動化されるかを体感しましょう。

2. Next.js (App Router) の基本構成

- `app/layout.tsx`: ページ共通の枠組み（全ページで使い回すもの）。
- `app/page.tsx`: 各ページのメインコンテンツ。
- `components/`: 自作した UI 部品を入れる場所。

3. Tailwind CSS の導入

CSS を別ファイルに書くのではなく、HTML の `class` に直接スタイルを指定する手法です。

```
// Before (Vanilla)
<div class="card">...</div>

// After (Tailwind)
<div className="bg-white p-6 rounded-xl shadow-md">...</div>
```

- **メリット:** クラス名を考える苦労がなくなり、爆速でコーディングできる。

4. 本日の目標: スケルトンの構築

- `create-next-app` でプロジェクトを初期化する。
- 第4週までの「静的な見た目」を Next.js + Tailwind CSS で再現する。
- ※まだ動かさない（JSの移行は来週以降）。まずは「見た目」を React の世界に持ってくる。

5. 躓きやすいポイント

1. **class** ではなく **className**: JSX (ReactのHTML的な書き方) ではクラス指定の単語が異なります。
2. 閉じタグ: **** や **
** も必ず **/>** で閉じる必要があります。
3. **TypeScript**: 最初はエラーが出まわるかもしれませんが、型 (Type) を定義することでバグを防げます。

宿題 (Homework)

1. `npx create-next-app@latest` を実行し、環境を構築する。
2. 第9週までのデザインを参考に、`page.tsx` を Tailwind CSS で装飾する。
3. 今後のコンポーネント分割に向けて、「どこが部品 (Component) になりそうか」を考えておく。